Maturitní projekt z informatiky

Martin Vanko

2025

1 Abstract

Vytvoření androidové aplikace, prozatím pouze pro mě, jejíž účel je pomoc při procesu memorizace šachových zahájení.

2 Úvod

Šachy jsem začal hrát v roce 2021. To byl pro mě dost důležitý rok, psal jsem taky přijímací zkoušky na GJK. Šachy jsem začal být doceal obsesovaný hned ze začátku, byla to aktivita ve které jsem viděl smysl. Nedělalo mi problém řešit tisíce šachovách úloh či čtení všemožných knih a studování partií starých mistrů, ale učení se zahájení. Všechny dostupné nástroje jsou určené k učení se ideí za tahy, ale neexistuje nástroj který by mi pomohl se samotnou memorizací zahájení.

3 Použité technologie

Většina kódu samotné aplikace je napsaná v Kotlinu. funkce na generování pseudo legálních tahů je v C. Queries do Sqlite3 databáze jsou napsané v SQL. Na popsání statické části jsem použil XML. Různé prvky např. šachovnici jsem jsem psal ve SVG formátu, což je taky druh XML. Programoval jsem v Android Studio od společnosti JetBrains a Visual Studio Code od Microsoftu.

3.1 XML

XML (*eXtensible Markup Language*) je markupový jazyk a zároveň souborový formát určený pro strukturované ukládání a přenos dat. V mém případě jsem psal layouty jednotlivých oken v XML. Výhodou XML je čitelnost nejen pro počítač, ale především pro uživatele.

3.2 Kotlin

Kotlin, podle ruského ostravu Kotlin, je moderní staticky typovaný programovací jazyk vyvinutý společností JetBrains, který běží na JVM (Java Virtual Machine) a je plně interoperabilní s Javou. Vybral jsem si ho, protože se nemusí psát tolik boilerplate kódu. Je to vlastně ve všem lepší Java.

3.3 C

C je nízkourovňový procedurální programovací jazyk vytvořený Dennisem Ritchiem v Bell Labs v 70. letech 20. století. Vybral jsem tento jazyk pro jednoduchost práce s pamětí a bitové manipulaci. Pro efektivní generování tahů jsem potřeboval precizní nástroj. Taky mám C velmi rád, neboť je krásně jednoduché.

3.4 SQL

SQL, nezkráceně *Structured Query Language*, je deklarativní jazyk pro správu relačních databází (jako SQLite3), standardizovaný organizací ISO/IEC. Využil jsem ho pro psaní dotazů do databáze.

3.5 SVG

SVG, nezkráceně *Scalable Vector Graphics*, je vektorový grafický formát založený na XML, specifikovaný W3C. Na začátku jsem dost řešil, jak uložit sprity, tlačítka, šachovnici atd. a došel jsem k tomu, že formát SVG je naprosto ideální. Pracuji s jednoduchými tvary a vyhnu se rozpixelovaným spritům, zároveň jsou modulární, například co se jednoduchosti změny barvby týče. Nějaké obrázky se mi nepovedlo najít na internetu, tak jsem je musel psát sám. Proto popisuji SVG jako zvolenou technologii.

3.6 Android Studio

Android Studio je integrované vývojové prostředí (IDE) od JetBrains určené pro tvorbu Androidových aplikací. Tato IDE byla pro mě vskutku esenciální. Velmi důležitý pro mě byl editor layoutů, který ukazoval v reálném čase provedené změny. A několikrát mě zachránila lokální historie.

3.7 Visual Studio Code

Visual Studio Code (VS Code) je multiplatformní textový editor od Microsoftu. Je docela lightweight. Dost tedy záleží na pohledu, oproti Android Studio je velmi lightweight, ale ve srovnání s Vimem určitě ne. Používal jsem ho výhradně pro napsání souboru pseudo_move_generation.h. Potom na vyzkoušení několika funkcí v Kotlinu.

4 Teoretická část

4.1 Pseudo_move_generation.h

Vždycky jsem si chtěl naprogramovat funkci generování tahů. Jde s tím dělat potom tolik věcí, co se šachu týče. Dlouho jsem přemýšlel, jak to uchopit, aby funkce byla rychlá a efektivní. Což pro tuto aplikaci úplně roli nehraje, ale do budoucna se to bude hodit a taky je to vlastně docela pěkné. Je spousta způsobů, jak šachovnici reprezentovat. Mohli bychom mít 32 figurek reprezentovaných pomocí barvy, typu a souřadnice na šachovnici. To je ale dosti neefektivní a, zavisící na implementaci, i dosti prostorově náročné. Problém tedy je jak, co nejefektivněji, víše zmíněné údaje reprezentovat. Já jsem pro řešení tohoto problému zvolil dvanáct šedesátičtyř bitových slov. Každý typ a barva figurky dostane vlastní slovo.

Pozice samotných figurek bude značená setnutými bity. A políčko a8 bude první bit a políčko h1 bude šedesátý čtvrtý bit. Pro samotnou reprezentaci slov jsem použil datový typ unsigned long long zkráceně ULL. V kódu protože unsigned long long nám nezaručí vždy 64 bitů.

```
typedef uint64_t Bitboard;
```

Kromě toho, že je to velmi prostorově úsporné, tak nám to zároveň umožní jednoduše provádět na slovech bitové operace.

Ukládání tahů Potom jsem potřeboval, jak uložit tahy. Jeden tah se dá popsat políčkem odkud, kam a potom nějakou značkou pro speciální tahy např. en passant nebo promoce.

```
/*
typedef int16_t Move;
0   0000   quiet moves
1   0001  double pawn push
2   0010   king castle
3   0011   queen castle
4   0100   captures
5   0101   ep-capture
8   1000   knight-promotion
9   1001   bishop-promotion
10  1010   rook-promotion
11  1011   queen-promotion
12  1100   knight-promo capture
13  1101   bishop-promo capture
14  1110   rook-promo capture
15  1111   queen-promo capture
*/
Move create_move(int flag, int from, int where){
    return ((((0x0|(where<<6))|from)|(flag<<12)));
}
```

2^6 je 64 tedy potřebujeme 6 bitů na reprezentaci políčka odkud a 6 bitů na reprezentaci políčka kam. Potom nám zbývají 4 bity na reprezentaci typu tahu.

FEN (Forsyth-Edwards Notation) definuje pozici tak, že řádek po řádku říká jaká figurka je na kterém políčku. Začíná na a8 a jde na h1. Čísla značí prádná políčka. Například:

- `starting_position_fen`: Standardní počáteční pozice. Která vypadá takto:

  ```
  "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR b KQkq - 0 1"
  ```

- `fen_test_castling`: Testovací pozice pro rošádu:

  ```
  "8/8/8/8/8/8/8/4K2R w K - 0 1"
  ```

- `fen_testing_ep_capture`: Testovací pozice pro braní mimochodem (en passant):

  ```
  "8/8/8/5pP1/8/8/8/8 w - f6 0 1"
  ```

Konstanty Potřeboval jsem spoustu konstant. Nejdůležitější jsou asi bitboardy typu: `not_a_file`, `not_h_file`, `not_eigth_rank`, neboť ty zaručují, že figurka nepřeteče na druhou stranu.

```
//0 1 1 1 1 1 1 1
//0 1 1 1 1 1 1 1
//0 1 1 1 1 1 1 1
//0 1 1 1 1 1 1 1
//0 1 1 1 1 1 1 1
//0 1 1 1 1 1 1 1
//0 1 1 1 1 1 1 1
//0 1 1 1 1 1 1 1
const Bitboard not_a_file = 0xfefefefefefefefeull;
//1 0 1 1 1 1 1 1
//1 0 1 1 1 1 1 1
//1 0 1 1 1 1 1 1
//1 0 1 1 1 1 1 1
//1 0 1 1 1 1 1 1
//1 0 1 1 1 1 1 1
//1 0 1 1 1 1 1 1
//1 0 1 1 1 1 1 1
const Bitboard not_b_file = 0xFDFDFDFDFDFDFDFDull;
Bitboard not_ab_file = ~(~not_a_file|~not_b_file);
Bitboard eastSoEa = ((knight << 10)&not_ab_file)&~ friendly_pieces;
```

Zabrání koni aby v tomto příkladě skončil na a1, což by byl ilegální tah.

```
//0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0
//0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0
//0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0
//0 0 0 0 0 0 0 0 -> //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0 -> //0 0 0 0 0 0 0 0
//0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0
//0 0 0 0 0 0 1 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 1 0 0    //0 0 0 0 0 0 0 0
//0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 1
//0 0 0 0 0 0 0 0    //1 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0    //0 0 0 0 0 0 0 0
               Illegal                                     Legal
```

Globální proměnné

- Bitboardy pro jednotlivé figury. Např.

  ```
  Bitboard white_king = 0ULL;
  Bitboard white_queen = 0ULL;
  Bitboard white_rook = 0ULL;
            ...
            ...
            ...
  ```

  Bitboardy začínají na nule a nastavím je až načtu FEN.

- Buffer pro ukládání tahů a počítadlo tahů:

```
    Move pseudo_moves[512];
    short pseudo_moves_counter = -1;
```

- Informace o Pozici:

```
    char turn = 0;
    unsigned char castling_rights = 0;
    Bitboard en_passant_square = 0ull;
```

Makra pro manipulaci s bity

- `bitset`, `bitclear`, `bitflip`, `bitcheck`: Operace s jednotlivými bity.

```
    #define bitset(bitboard, nbit) ((bitboard) |= (1ull<<(nbit)))
    #define bitclear(bitboard,nbit) ((bitboard) &= ~(1ull<<(nbit)))
    #define bitflip(bitboard,nbit) ((bitboard) ^= (1ull<<(nbit)))
    #define bitcheck(bitboard,nbit) ((bitboard) & (1ull<<(nbit)))
```

Definice směrů a typů tahů

- Směry pohybu:

```
#define EAST 1
#define NORTHEAST 2
#define SOUTHEAST 3
#define WEST 4
    ...
    ...
    ...
```

- Typy tahů (flagy): `QUIET_MOVES` (standardní tah), `CAPTURES` (sebrání figury), `KING_CASTLE` (královská rošáda), `EP_CAPTURE` (en passant), atd.

```
#define QUIET_MOVES 0
#define DOUBLE_PAWN_PUSH 1
#define KING_CASTLE 2
#define QUEEN_CASTLE 3
#define CAPTURES 4
        ...
        ...
        ...
```

Hlavní funkce pseudo_moves_generator(char* fen) nejdříve načte FEN notaci a vytvoří tak jednotlivé bitboardy. Potom vytvoří:

```
Bitboard whitePieces =
    white_king|white_queen|white_rook|white_knight|white_bishop|white_pawn;
Bitboard blackPieces =
    black_king|black_queen|black_rook|black_knight|black_bishop|black_pawn;
```

Pro kontrolu jestli, bílá figurka nebere druhou bílou figurku. Což by bylo ilegální. Potom jsem si vytvořil funkce pro jednotlivé figurky.

```
static Bitboard pseudo_king_moves(Bitboard king, Bitboard friendly_pieces, Bitboard
    opponent_pieces);
static Bitboard pseudo_queen_moves(Bitboard queen, Bitboard friendly_pieces, Bitboard
    opponent_pieces);
static Bitboard pseudo_bishop_moves(Bitboard bishop, Bitboard friendly_pieces, Bitboard
    opponent_pieces);
static Bitboard pseudo_rook_moves(Bitboard rook, Bitboard friendly_pieces, Bitboard
    opponent_pieces);
static Bitboard pseudo_knight_moves(Bitboard knight, Bitboard friendly_pieces, Bitboard
    opponent_pieces);

static Bitboard pseudo_white_pawn_moves(Bitboard pawn, Bitboard friendly_pieces, Bitboard
    opponent_pieces);
static Bitboard pseudo_black_pawn_moves(Bitboard pawn, Bitboard friendly_pieces, Bitboard
    opponent_pieces);
```

Projdu skrze všechny bitboardy, vytvořím tahy a přidám je do bufferu. Nakonec si tahy překopíruji z bufferu do pole, kde první element je počet tahů.

```
Move * moves = (Move*)malloc((pseudo_moves_counter+2) * sizeof(Move));
                          ...
```

4.2 JNI

V srdci celé aplikace leží komunikace mezi knihovou v C a Kotlinem. Toho jsem se na začátku bál nejvíce, neboť JNI je hrozné. JNI je zkratka pro Java Native Interface, neboli způsob jak volat nativní kód z Javy. Dává mi jen smysl projít si můj "interface".

```
#include <jni.h>
#include <cstring>
#include "pseudo_move_generation.h"

char* jstringToChar(JNIEnv* env, jstring jstr) {
    const char* c_str = env->GetStringUTFChars(jstr, nullptr);
    if (c_str == nullptr) {
        return nullptr;
    }
    char* copy = new char[strlen(c_str) + 1];
    strcpy(copy, c_str);
    env->ReleaseStringUTFChars(jstr, c_str);
    return copy;
```

```
}

extern "C"
JNIEXPORT jshortArray JNICALL
Java_com_martinsapps_chessproject_ChessBoard_getPseudoMoves(
        JNIEnv *env,
        jobject,
        jstring fenString) {

    char* fen = jstringToChar(env, fenString);
    if (fen == nullptr) {
        return nullptr;
    }

    short * moves = pseudo_moves_generator(fen);

    int num_moves = moves[0]+1;

    jshortArray java_array = env->NewShortArray(num_moves);
    env->SetShortArrayRegion(java_array, 0, num_moves, moves);

    delete[] fen;
    delete[] moves;
    return java_array;
}
```

ZBÝVÁ DOPSAT

4.3 Databáze

Pro ukládání dat jsem zvolil SQLite3 databázi. Pro můj účel by určitě bohatě postačil textový soubor, co se optimalizace týče. Přijde mi ale SQL psát praktické a už jsem s tím pracoval.

4.3.1 Kopírování databáze

Pro uložení všech zahájení a pak dalších metadat jsem použil SQLite3 databázi. Nejdříve musím databázi zkopírovat do zařízení, pokud je to první otevření.

```
private fun copyDataBase() {
        try {
            val mInput: InputStream = applicationContext.assets.open("openings.db")
            val outFileName: String =
                "/data/data/com.martinsapps.chessproject/databases/" + "openings.db"
            val mOutput: OutputStream = FileOutputStream(outFileName)
            val mBuffer = ByteArray(1024)
            var mLength: Int
            while (mInput.read(mBuffer).also { mLength = it } > 0) {
                mOutput.write(mBuffer, 0, mLength)
            }
            mOutput.flush()
```

```
            mOutput.close()
            mInput.close()
        } catch (e: IOException) {
            println(e)
        }
    }
```

Pro komunikaci s databází jsem zvolil balíček android.database.sqlite.SQLiteOpenHelper a napsal jsem si třídu, která extenduje SQLiteOpenHelper a vypadá takto:

```
class DbHandler
    (context: Context?, name: String?, factory: SQLiteDatabase.CursorFactory?, version:
        Int) :
    SQLiteOpenHelper(context, name, factory, version)
```

Potom jsem si napsal jednotlivé funkce na exekutování dotazů.

4.3.2 .SCHEMA

Samotné schema databáze vypadá takto:

```
CREATE TABLE fried_liver (move_number INTEGER PRIMARY KEY, fen TEXT NOT NULL);
CREATE TABLE najdorf (move_number INTEGER PRIMARY KEY, fen TEXT NOT NULL);
CREATE TABLE giuoco_piano (move_number INTEGER PRIMARY KEY, fen TEXT NOT NULL);
...
...
...
CREATE TABLE settings (chessboard TEXT, legal_moves INT, sound_effects INT);
CREATE TABLE last_played (table_name TEXT, color INT, name TEXT);
CREATE TABLE streak (streak INT);
```

4.4 Activity

Každá aktivita je jedno okno, co uvidí uživatel. Má vlastní layout.xml soubor, kde je definovaný její layout. Potom má každá svůj soubor JménoAktivity.kt, což je třída, která dědí ze super třídy AppCompatActivity.

```
    class MainActivity : AppCompatActivity()
```

V naší třídě potom overridnem různé metody z rodičovské třídy AppCompatActivity. Nejdůležitější taková metoda je

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

Kde argument savedInstanceState je poslední stav aktivity. Další důležitou metodou by byla, alespoň pro mne metoda finish(), díky které mám skvělé animace.

```
override fun finish() {
        super.finish()
        overridePendingTransition(R.anim.popup_enter, R.anim.popup_exit)
    }
```

4.4.1 SplashActivity

To je aktivita, která v sobě skrývá pouze úvodní animaci. Otevře se jako první, provede animaci popsanou níže.

Kus kódu z SplashActivity.kt

```
val animation = AnimationUtils.loadAnimation(this, R.anim.logo_enter)
logo.startAnimation(animation)
```

Kód logo_enter.xml

```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android"
    android:fillAfter="true">

    <!-- Fade in effect -->
    <alpha
        android:duration="500"
        android:fromAlpha="0.0"
        android:toAlpha="0.6" />

    <!-- Scale up effect -->
    <scale
        android:duration="500"
        android:fromXScale="0.5"
        android:fromYScale="0.5"
        android:toXScale="0.9"
        android:toYScale="0.9"
        android:pivotX="50%"
        android:pivotY="50%" />
</set>
```

4.4.2 MainActivity

Každý aplikace potřebuje nějakou hlavní aktivitu. Jedná se o první aktivitu, který se otevře. Samozřejmě jde přejmenovat. A shodou okolností moje první aktivita je SplashActivity. Moje hlavní aktivita obsahuje jen komunikaci s databází a dále slouží jako rozcestník. Jediný zajímavý kus kódu je asi práce se streak ohýnkem.

Streak Streak vždy začíná jedničkou. S tím jsem měl docela problém, neboť jsem zastánce toho, že by se věci měli počítat od nuly. Třeba čas počítáme od nuly, měsíce nikoliv a je v tom nepořádek. Řekl jsem si ale, že budoucímu uživateli určitě udělá větší radost streak s hodnotou jedna. Ačkoliv mně to radost neudělalo.

```
if (daysBetween == 1L){
    var streakCurrent = dbHandler.getStreak()
    streakCurrent +=1
    val milliseconds = System.currentTimeMillis()
    dbHandler.updateStreak(streakCurrent, milliseconds)
}
else if (daysBetween > 1){
    val milliseconds = System.currentTimeMillis()
    dbHandler.updateStreak(1, milliseconds)
}else if (daysBetween < 0){
    val milliseconds = System.currentTimeMillis()
    dbHandler.updateStreak(1, milliseconds)
}
```

Kód pro popis chování streaku. Myslel jsem samozřejmě na edge case, kdy by si uživatel přetočil čas. Rád jsem jako dítě takto podváděl v hrách.
Asi poslední zajímavá věc je, jak se mění obrázek plamene s rostoucím streakem. Vytvořil jsem si funkci:

```
changeSvgFillColor(streak, R.drawable.streak_flame, 255,255,255, (255 * (1 - exp((-0.05*
    currentStreak)))).toInt())
```

Která bere jako argument instanci ImageView objektu, sprite objektu a hodnoty RGB plus aplha. Rozhodl jsem se, že nebudu měnit barvu plamene z estetických důvodů, ačkoliv jsem to už naprogramoval a budu měnit pouze alphu. Alpha říká, jak je ImageView průhlednej a argument může být z otevřeného intervalu celých čísel $(0, 255)$. Chtěl jsem tedy posloupnost, která bude mít nevlastní limitu $\lim_{n\to\infty} a_n = 255$ a zvolil jsem $255 \times (1 - e^{-k \times currentStreak})$, kde k je nějaký parametr určující, jak rychle funkce poroste. $k \in \mathbb{R}$, ale ideální hodnoty pro můj účel jsou z intervalu $(0, 1)$.

4.4.3 OpeningsRoot

Toto je rozcestník pro výběr zahájení. V main activity jsou dvě ImageButtons s kódem:

```
val whiteOpenings = findViewById<ImageButton>(R.id.white)
       whiteOpenings.setOnClickListener {
           val intent = Intent(this, OpeningsRoot::class.java)
           intent.putExtra("color", 0)
           this.startActivity(intent)
           finish()
       }
       val blackOpenings = findViewById<ImageButton>(R.id.black)
       blackOpenings.setOnClickListener {
           val intent = Intent(this, OpeningsRoot::class.java)
           intent.putExtra("color", 1);
           this.startActivity(intent)
           finish()
       }
```

Kód akorát říká, že po kliknutí otevřu OpeningsRoot activity a přidám tam extra informaci o barvě. Prostě jestli si vybírám ze zahájení pro bílého nebo černého. Nic moc zajímavého se zde neděje.

Akorát vytvářím ImageButton pro každé zahájení. A kdyby náhodou zahájení nemělo jméno nebo obrázek, tak mám nějaký stock text.

4.4.4 Openings

Zde kreslím figurky a šachovnici. Figurky kreslím zvlášť dle vybrané barvy. Stejně jako OpeningsRoot přišel s nějakou informací navíc, tak zde mám:

```
val color = intent.getIntExtra("color", 0)
val dbOpeningName = intent.getStringExtra("opening")
val openingName = intent.getStringExtra("name")
```

dbOpeningName je jméno zahájení v databázi a openingName je jméno s mezerami. Takto počítám začátek šachovnice:

```
val screenRatio = screenWidth.toFloat()/screenHeight.toFloat()
val chessB\endoardStartY= round((screenHeight/(screenRatio*7)))
```

Což nebude fungovat na hodně čtvercových displejích, ale pro ty budu muset stejně vytvořit samostatný layouty.

4.5 Třídy

4.5.1 ChessBoard

Třídu `ChessBoard` používám jako komunikátora mezi figurkami a zbytkem. Každá figurka dostane jako argument instanci šachovnice a to samé většina dalších tříd.

Vlastnictví

- `val squareCoordinates: Array<IntArray>`: Je matice všech 64 polí a jejich souřadnic na obrazovce.
- `val squareSize:Int`: Velikost jednoho políčka v pixelech.
- `var fen: String`: Zápis aktuální pozice v FEN viz výše.
- `var pieces: MutableList<ImageView>`: List `ImageView` objektů, které používám jako figurky.
- `var turn: Boolean`: Určuje, zda je na tahu bílý (`true`) nebo černý (`false`).
- `var plyCounter: Int`: Jedno ply je půl tah. Když bílý zahraje v prvním tahu e4 tak ply je 1. Kdybych používal pouze tahy, tak bych přišel o info, jestli jsme v půli tahu nebo na konci.
- `castling rights`:

  ```
  var whiteKingsideCastle = true
  var whiteQueensideCastle = true

  var blackKingsideCastle = true
  var blackQueensideCastle = true
  ```

Práva na rošádu a jejich permutace.

- `var enPassantSquare = false`: Říka souřadnice pole, kde je možný en passant.

Metody

- `generateFen(color: Int): String`
  Vytvoří FEN z pole pieces a vrátí ho jako String. Funkce to moc hezká není, ale funguje skvěle.
- `getPlayedMove(fen1: String, fen2: String): IntArray`
  Detekuje provedený tah mezi dvěma FEN pozicemi. Používám na zobrazení hintu. Vrací pole Integerů`[odkud, kam]`. Stejně jako můj datatyp Move.
- `getPlayedMoveAlgebraicWithPiece(fen1: String, fen2: String): String`
  Dělá to samé, ale vrací místo [36, 53] e4 pro tah pěšec z e2 na e4.
- `pseudoLegalMoves(fen: String): MutableList<Move>`
  Získává pseudolegální tahy pomocí nativní funkce `getPseudoMoves`. Pro rošádu a speciální tahy využívám flagy viz výše.
- `drawPieces(...)`
  Vykreslí figurky na šachovnici podle FEN zápisu. Zase pro obě strany zvlášť.

4.5.2 ChessPiece

Třída `ChessPiece` popisuje jednotlivé figurky, především jejich vizuální stránku, logika je ošetřená jinde. Asi nejdůležitější částí je logika toho, co se stane po kliknutí na jednotlivé figurky.

Vlastnictví

- `ID`: Každá figurka má vlastní ID, které odpovídá číselným id zdrojům spritů figurek (např. `R.drawable.black_king`).
- `imageView`: Samotný obrázek, který je vidět na obrazovce. Taky tohle je ten ImageView, který je v poli u ChessBoard.
- `chessBoard`: Každé figurce jsem strčil jako argument instanci šachovnice, abych si mohl šáhnout například na políčka nebo další atributy.

Metody

- `drawPiece(x: Float, y: Float, width: Int, height: Int): ImageView`
  Vytvoří `ImageView` s figurou na zadaných souřadnicích a nastaví její výšku a šířku. Figurky jsou čtvercového tvaru a pak je stejně kreslím takto:

```
val piece = createPiece(ChessPiece.BLACK_ROOK).drawPiece(currentSquareX,
    currentSquareY, pieceSize, pieceSize)
```

- `makePieceClickable()`
  Nastaví klikací události pro figurku. Určitě se jedná o největší monstrozitu tohoto projektu.:
  - Po kliknutí zvýrazní políčko zeleným čtvercem. Pokud to nastavení dovoluje.
  - Zobrazí dostupné tahy pomocí `GreenSquareFactory`. Pokud to nastavení dovoluje.
  - Když kliknu na políčko, tak zkontroluju, jestli je v legálních tazích. Potom aktualizuji pozici a FEN v ChessBoard.
  - Udělám zvuk pomocí třídy SoundPlayer. Např.

```
when(moveType){
    1->chessBoard.soundPlayer.playMoveSound()
    2->chessBoard.soundPlayer.playCaptureSound()
}
```

4.5.3 GreenSquareFactory

Třída `GreenSquareFactory` je pro zobrazování teček pro ukázání legálních tahů, čtverců různých barev pro označení hintu, správných a špatných tahů.

Vlastnictví

- `greenSquareList`: Seznam `ImageView` prvků reprezentujících vizuální efekty (tečky, kroužky, čtverce). Jediný účel je, že je musím být schopen smazat. Musím si tak uložit nějaký odkaz.
- `chessBoard`: Odkaz na instanci šachovnice pro přístup k souřadnicím políček.

Metody

- `addHollowCircle(squareNumber: Int)`
  Přidá průhledný zelený kroužek na políčko s daným indexem. Používám ho pro označení captures a rošády. Argument je číslo políčka, pomocí ChessBoard zjistím souřadnice.
- `addDot(squareNumber: Int)`
  Přidá zelenou tečku na políčko, kde je pseudo legální tah. Zobrazuje se pouze pokud je v nastavení (`legal_moves`) povoleno zobrazování tahů. Argument je číslo políčka, pomocí ChessBoard zjistím souřadnice.
- `addRedSquare(coordinates: FloatArray)`
  Vytvoří červený čtverec na zadaných souřadnicích. Přidal jsem ho, abych dal uživateli najevo, že zahájení pokazil. Čtverec postupně mizí pomocí animace `fadeOut`.
- `addGreenSquare(squareNumber: Int)`
  Využívám pro zvýraznění vybrané figurky nebo hintu. Argument je číslo políčka, pomocí ChessBoard zjistím souřadnice.
- `removeSquares()`
  Odstraní všechny aktivní vizuální efekty z layoutu a vyčistí `greenSquareList`.

4.5.4 SoundPlayer

Třída `SoundPlayer` je k přehrávání zvukových efektů. Používám balíček:

```
android.media.SoundPool
```

Hlavní atributy

Potom akorát načtu efekty. Jinak se to seká.

```
private fun preloadSounds(context: Context) {
        moveSound = soundPool.load(context, R.raw.move, 1)
        captureSound = soundPool.load(context, R.raw.capture, 1)
        wrongSound = soundPool.load(context, R.raw.wrong, 1)
        goodSound = soundPool.load(context, R.raw.good, 1)
        soundPool.setOnLoadCompleteListener { _, sampleId, status ->
            if (status == 0) {
                soundPool.play(sampleId, 0f, 0f, 0, 0, 1.0f)
            }
        }
    }
    fun playMoveSound() {
        val settings = dbHandler.getSettings() ?: throw IOException()
        val soundEffects = settings["sound_effects"].toString().toInt()
        if (soundEffects == 1) {
            soundPool.play(moveSound, 1.0f, 1.0f, 0, 0, 1.0f)
        }
    }
```

- `soundPool`: Instance `SoundPool` je to, co reálně přehrává ty zvuky.
- `dbHandler`: Odkaz na databázi pro kontrolu nastavení zvukových efektů.

5 Návod k použití

Je potřeba mít přístup k nějakému zařiážení na kterém běží android. Vytvořil jsem release na githubu s url: https://github.com/Martin49232/chessProject/releases. Stačí si stáhnout soubor openingo.apk a nainstalovat si ho. Nebo si projekt může uživatel postavit sám skrze například výše zmíněné Android Studio.

Po nainstalování zapněte aplikaci. Otevře se hlavní okno a uživatel dostane na výběr ze zahájení pro černého a bílého. Po vybrání strany se dostane do okna pro výběr samotného zahájení. Po vybrání zahájení se otevře učící program.

6 Ukázka z aplikace

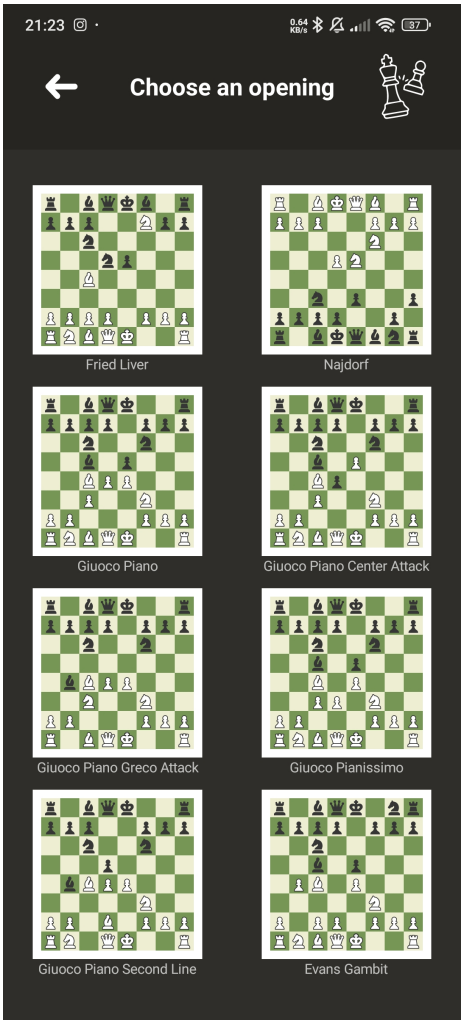

Figure 1: Aktivita OpeningsRoot

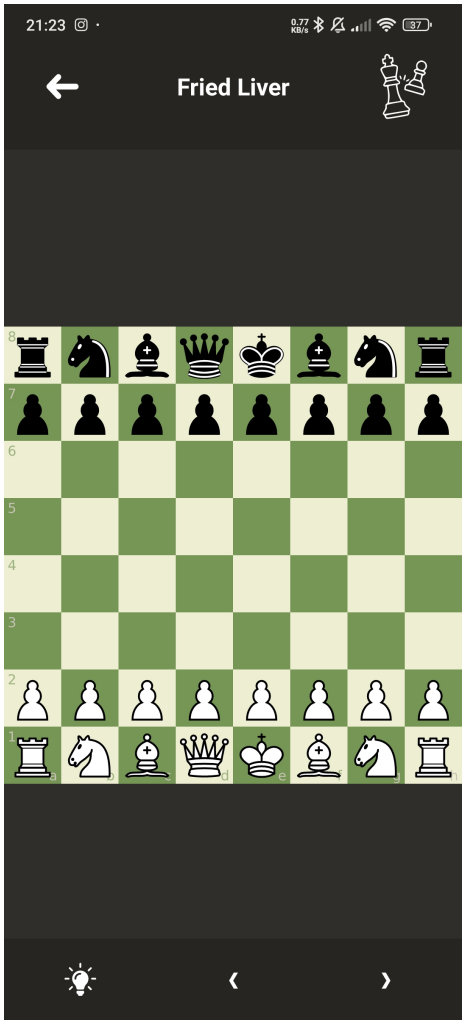

Figure 2: Aktivita Openings

7 Závěr

Povedlo se mi splnit a implementovat vše, co jsem si zadal. Důležité je znovu zdůraznit, že aplikaci jsem vytvořil především pro sebe, není proto optimalizovaná pro displeje obskurnějších tvarů. Aplikace se spouští bez chyb a na žádný nevyřešitelný bug jsem ještě nepřišel.